Übungsblatt 5 – Lösungen

Ableitung

Teil A: Auf Papier

Aufgabe A1 („Schließlich“ und Klein-o)

Zur Erinnerung: $f = o(g)$ längs eines Filters l heißt, dass zu *jedem* $c > 0$ schließlich $|f(x)| \leq c\,|g(x)|$ gilt; bei $f = O(g)$ genügt *ein* c. Ist $g(x) \neq 0$ schließlich, so gilt $f = o(g)$ genau dann, wenn $f/g \to 0$, und $f = O(g)$ genau dann, wenn f/g schließlich beschränkt ist. (Bei $\mathcal{N}(0)$ rechnet man mit $x \neq 0$; die Funktionen hier verschwinden in 0, dort stimmt die Ungleichung von selbst.)

(a) Geben Sie an:

(i) das kleinste ganze N, so dass $x^2 \geq 100\,x$ für alle $x \geq N$ gilt;

(ii) das kleinste ganze N, so dass $3x + 7 < x^2$ für alle $x \geq N$ gilt;

(iii) das kleinste ganze N, so dass $1/x \leq 1/1000$ für alle $x \geq N$ gilt;

(iv) ein $\delta > 0$, so dass $x^2 > 3$ für alle x mit $|x - 2| < \delta$ gilt.

(b) Entscheiden Sie mit dem Quotienten f/g, ob $f = o(g)$ und ob $f = O(g)$ gilt:

	f	g	längs	$f = o(g)$?	$f = O(g)$?
1	x^2	x	$\mathcal{N}(0)$		
2	x^2	x	atTop		
3	$\sin x$	x	$\mathcal{N}(0)$		
4	$\sin x$	x	atTop		
5	$x^2 + x$	x^2	atTop		
6	$x^2 + x$	x^2	$\mathcal{N}(0)$		

(c) Zu jedem c gehört ein eigenes δ bzw. N:

(i) $x^3 = o(x^2)$ längs $\mathcal{N}(0)$: Geben Sie zu $c = \frac{1}{10}$ und zu $c = \frac{1}{100}$ ein δ an, so dass $|x^3| \leq c\,x^2$ für $|x| < \delta$.

(ii) $x^2 = o(x^3)$ längs atTop: Geben Sie zu $c = \frac{1}{10}$ und zu $c = \frac{1}{100}$ ein N an, so dass $x^2 \leq c\,|x^3|$ für $x \geq N$.

(iii) x^3 ist längs atTop *nicht* $o(x^2)$: Für welche x gilt $|x^3| \leq 1 \cdot x^2$, und warum ist das kein Endstück?

(d) Zeigen Sie $2x^2 + 7x^3 = o(x)$ längs $\mathcal{N}(0)$, erst mit dem Quotienten, dann so: $x^2 = o(x)$ und $x^3 = o(x)$ gelten längs $\mathcal{N}(0)$. Mit welchem c' prüft man beide Summanden, damit die Summe zu $c = \frac{1}{10}$ höchstens $c\,|x|$ wird?

Hinweis: Zu (a): Kleinstes N heißt, dass die Ungleichung bei $N - 1$ schon nicht mehr für alle größeren x gilt. Zu (c, iii): $x^3 \leq x^2$ heißt $x \leq 1$.

Lösung. **(a)** (i) $x^2 - 100x = x\,(x - 100) \geq 0$ für $x \geq 100$, aber negativ für $0 < x < 100$: $N = 100$. (ii) $x^2 - 3x - 7 > 0$ gilt genau für $x > \frac{3+\sqrt{37}}{2} \approx 4{,}54$ (und für große negative x), also $N = 5$; bei $x = 4$ ist $16 < 19$. (iii) Für $x > 0$ ist $\frac{1}{x} \leq \frac{1}{1000} \iff x \geq 1000$: $N = 1000$. (iv) Für $x > 0$ ist $x^2 > 3 \iff x > \sqrt{3} \approx 1{,}73$. Für $|x - 2| < \frac{1}{4}$ ist $x > \frac{7}{4}$, also $x^2 > \frac{49}{16} > 3$: $\delta = \frac{1}{4}$ geht. Am größten darf δ mit $2 - \sqrt{3} \approx 0{,}27$ sein.

(b)

	Quotient	$f = o(g)$?	$f = O(g)$?
1	$\frac{x^2}{x} = x \to 0$ für $x \to 0$	ja	ja
2	$\frac{x^2}{x} = x \to \infty$ für $x \to \infty$	nein	nein
3	$\frac{\sin x}{x} \to 1$ für $x \to 0$	nein	ja
4	$\left\lvert\frac{\sin x}{x}\right\rvert \leq \frac{1}{\lvert x\rvert} \to 0$ für $x \to \infty$	ja	ja
5	$\frac{x^2+x}{x^2} = 1 + \frac{1}{x} \to 1$ für $x \to \infty$	nein	ja
6	$\left\lvert 1 + \frac{1}{x}\right\rvert \to \infty$ für $x \to 0$	nein	nein

Zeile 3 und 4 haben dieselben Funktionen und verschiedene Antworten: Der Filter entscheidet mit. Ist der Quotient gegen eine Konstante $\neq 0$ konvergent (Zeilen 3 und 5), so ist $f = O(g)$, aber nicht $o(g)$.

(c) (i) Für $|x| < c$ ist $|x^3| = |x| \cdot x^2 \leq c\,x^2$: $\delta = c$, also $\delta = \frac{1}{10}$ bzw. $\frac{1}{100}$. (ii) Für $x \geq \frac{1}{c}$ ist $cx \geq 1$, also $x^2 \leq cx \cdot x^2 = c\,x^3$: $N = \frac{1}{c}$, also $N = 10$ bzw. $N = 100$. Ein kleineres c verlangt ein kleineres δ bzw. ein größeres N – zu jedem c eine eigene große Menge. (iii) Für $x \neq 0$ ist $|x|^3 \leq x^2 \iff |x| \leq 1$; die Menge $S_1 = [-1, 1]$ enthält kein Endstück $[N, \infty)$, denn $\max(N, 2)$ liegt nicht in ihr: Für $x \geq 2$ ist $x^3 \geq 2x^2 > x^2$. Dieselben zwei Funktionen liefern bei 0 und bei ∞ die umgekehrte Aussage.

(d) Der Quotient ist $\frac{2x^2+7x^3}{x} = 2x + 7x^2 \to 0$ für $x \to 0$. Mit der Definition: Zu $c = \frac{1}{10}$ nimmt man für beide Summanden $c' = \frac{c}{|2|+|7|+1} = \frac{1}{100}$. Auf dem Schnitt der zwei großen Mengen gilt $|x^2| \leq c'\,|x|$ und $|x^3| \leq c'\,|x|$, also

$$|2x^2 + 7x^3| \leq 2\,|x^2| + 7\,|x^3| \leq (2 + 7)\,c'\,|x| \leq 10\,c'\,|x| = c\,|x|.$$

Der Schnitt ist nach Axiom (iii) groß, die Menge, auf der die Summe abgeschätzt ist, enthält ihn, ist also nach Axiom (ii) groß. Das „+1“ im Nenner erspart den Sonderfall $a = b = 0$.

Aufgabe A2 (Die Ableitung)

Nach der Vorlesung hat f in x die Ableitung f', wenn der Fehler der Tangente, $f(y) - f(x) - (y - x)\,f'$, klein gegen $y - x$ ist, also $o(y - x)$ für $y \to x$.

(a) Bestimmen Sie die Tangente an $f(y) = y^2$ in $x = 3$, und rechnen Sie den Fehler der Tangente aus. Wie groß ist er bei $y = 3{,}1$? Für welche y ist er höchstens $\frac{1}{100}\,|y - 3|$?

(b) Zeigen Sie aus der Definition: $f(y) = y^2 + 3y$ hat in x die Ableitung $2x + 3$. Machen Sie die Probe bei $x = 1$, $y = \frac{6}{5}$.

(c) Sei $g(y) = |y|$. Berechnen Sie die Sekantensteigungen $\frac{g(y)-g(0)}{y-0}$ für $y = \pm 0{,}1$ und $y = \pm 0{,}01$. Hat g in 0 eine Ableitung? Hat g in 0 ein lokales Minimum? Widerspricht das dem Satz von Fermat?

Hinweis: In (b) lässt sich der Fehler als Quadrat schreiben, wie in (a).

Lösung. **(a)** Mit $f'(3) = 6$ ist die Tangente $y \mapsto 9 + 6\,(y-3)$. Der Fehler ist

$$y^2 - 9 - 6\,(y-3) = y^2 - 6y + 9 = (y-3)^2.$$

Bei $y = 3{,}1$: $f = 9{,}61$, Tangente 9,6, Fehler $0{,}01 = (0{,}1)^2$. Der Fehler $(y-3)^2$ ist höchstens $c\,|y-3|$, wenn $|y-3| \le c$ ist; zu $c = \frac{1}{100}$ also für $|y-3| \le \frac{1}{100}$.

(b) Der Fehler der Tangente mit Steigung $2x+3$ ist

$$\begin{aligned} y^2 + 3y - x^2 - 3x - (y-x)(2x+3) &= (y-x)(y+x) + 3(y-x) - (y-x)(2x+3) \\ &= (y-x)\,(y-x) = (y-x)^2, \end{aligned}$$

und das ist wie in (a) höchstens $c\,|y-x|$ für $|y-x| \le c$: also $o(y-x)$. Probe: $f(1) = 4$, $f'(1) = 5$, $f(\frac{6}{5}) = \frac{36}{25} + \frac{18}{5} = \frac{126}{25} = 5{,}04$; die Tangente liefert $4 + 5 \cdot \frac{1}{5} = 5$; der Fehler ist $0{,}04 = (\frac{1}{5})^2$.

(c) Für $y > 0$ ist die Sekantensteigung $\frac{|y|}{y} = 1$, für $y < 0$ ist sie -1; bei $y = \pm 0{,}1$ und $\pm 0{,}01$ also abwechselnd $+1$ und -1. Ein Grenzwert für $y \to 0$ existiert nicht: g hat in 0 keine Ableitung. Mit der Steigung 0 wäre der Fehler der Tangente $|y|$, und $|y| \le \frac{1}{2}\,|y|$ gilt für kein $y \neq 0$: $|y|$ ist nicht klein gegen y. g hat in 0 aber ein (sogar globales) Minimum, $|y| \ge 0 = g(0)$. Das widerspricht Fermat nicht: Fermat sagt, dass $f'(a) = 0$ ist, *wenn* die Ableitung existiert. Hier gibt es keine Tangente, die waagerecht sein könnte.

Aufgabe A3 (Partielle Ableitungen, Gradient, Hesse-Matrix)

Sei $f(x,y) = x^2 y + 3y^2$.

(a) Berechnen Sie $\dfrac{\partial f}{\partial x}$ und $\dfrac{\partial f}{\partial y}$ und den Gradienten $\nabla f(1,2)$.

(b) Berechnen Sie die vier zweiten partiellen Ableitungen und die Hesse-Matrix in $(1,2)$. Was fällt bei den gemischten Ableitungen auf?

(c) Berechnen Sie $f(1,2) + \nabla f(1,2) \cdot h$ für $h = (\frac{1}{10}, -\frac{1}{10})$ und vergleichen Sie mit $f(1{,}1;\ 1{,}9)$. Wie groß ist die Richtungsableitung von f in $(1,2)$ in Richtung $v = (1,-1)$?

(d) Sei $k(x,y) = \dfrac{xy}{x^2+y^2}$ für $(x,y) \neq (0,0)$ und $k(0,0) = 0$. Berechnen Sie $k(t,0)$, $k(0,t)$ und $k(t,t)$ für $t \neq 0$. Was folgt für die partiellen Ableitungen in $(0,0)$, und ist k in $(0,0)$ stetig? Ist k dort differenzierbar?

Hinweis: Bei einer partiellen Ableitung hält man die anderen Variablen fest und leitet in einer Variablen ab. Zu (c): $Df(x)\,h = \nabla f(x) \cdot h$.

Lösung. **(a)** $\dfrac{\partial f}{\partial x} = 2xy$ (hier ist y eine Konstante), $\dfrac{\partial f}{\partial y} = x^2 + 6y$. Im Punkt $(1,2)$: $\nabla f(1,2) = (2 \cdot 1 \cdot 2,\ 1 + 12) = (4,\ 13)$.

(b) $f_{xx} = 2y$, $f_{xy} = 2x$, $f_{yx} = 2x$, $f_{yy} = 6$, also

$$\begin{pmatrix} 2y & 2x \\ 2x & 6 \end{pmatrix} \quad \text{und in } (1,2): \quad \begin{pmatrix} 4 & 2 \\ 2 & 6 \end{pmatrix}.$$

Die gemischten Ableitungen f_{xy} und f_{yx} sind gleich; die Hesse-Matrix ist symmetrisch. (Das gilt, wenn die zweiten Ableitungen stetig sind.)

(c) $f(1,2) = 2 + 12 = 14$ und $\nabla f(1,2) \cdot h = 4 \cdot 0{,}1 + 13 \cdot (-0{,}1) = -0{,}9$, also die Näherung 13,1. Exakt ist $f(1{,}1;\ 1{,}9) = 1{,}21 \cdot 1{,}9 + 3 \cdot 3{,}61 = 2{,}299 + 10{,}83 = 13{,}129$; der Fehler ist 0,029. Mit $\frac{h}{10}$ statt h ist er nur noch etwa 0,0003 – hundertmal kleiner bei zehnmal kürzerem Schritt: Der Rest ist klein gegen h. Die Richtungsableitung ist $Df(1,2)\,v = \nabla f(1,2) \cdot v = 4 - 13 = -9$: Entlang $(1,-1)$ fällt f mit der Steigung -9.

(d) $k(t,0) = 0$ und $k(0,t) = 0$ für $t \neq 0$, und $k(t,t) = \frac{t^2}{2t^2} = \frac{1}{2}$. Die Schnittkurven längs der Achsen sind konstant 0, also sind beide partiellen Ableitungen in $(0,0)$ gleich 0. Auf der Diagonalen ist $k = \frac{1}{2}$ für alle $t \neq 0$, aber $k(0,0) = 0$: k ist in $(0,0)$ nicht stetig. Also ist k dort nicht differenzierbar (aus Differenzierbarkeit folgt Stetigkeit) – obwohl beide partiellen Ableitungen existieren. Partielle Ableitungen allein reichen nicht.

Aufgabe A4 (Jacobi-Matrix und Kettenregel)

Seien $F\colon \mathbb{R}^2 \to \mathbb{R}^2$, $F(x,y) = (x^2 - y^2,\ 2xy)$ und $g\colon \mathbb{R}^2 \to \mathbb{R}$, $g(u,v) = u^2 + v^2$.

(a) Berechnen Sie die Jacobi-Matrix $J_F(x,y)$ und ihre Determinante. Was sind beide in $(1,2)$?

(b) Berechnen Sie $F(1,2) + J_F(1,2)\,h$ für $h = (\frac{1}{10}, -\frac{1}{10})$ und vergleichen Sie mit $F(1{,}1;\ 1{,}9)$.

(c) Berechnen Sie $J_{g\circ F}(1,2)$ mit der Kettenregel, $J_g(F(1,2)) \cdot J_F(1,2)$, und noch einmal direkt, nachdem Sie $g(F(x,y))$ vereinfacht haben.

(d) Seien $P(r,\varphi) = (r\cos\varphi,\ r\sin\varphi)$ und g wie oben. Berechnen Sie $J_P(2, \frac{\pi}{2})$ mit seiner Determinante und $J_{g\circ P}(2, \frac{\pi}{2})$ mit der Kettenregel. Direkt: Was ist $g(P(r,\varphi))$?

Hinweis: Die Zeilen der Jacobi-Matrix sind die Gradienten der Komponenten. Zu (c): $(x^2-y^2)^2 + (2xy)^2 = (x^2+y^2)^2$.

Lösung. (a)

$$J_F(x,y) = \begin{pmatrix} 2x & -2y \\ 2y & 2x \end{pmatrix}, \qquad \det J_F = 4x^2 + 4y^2 = 4\,(x^2+y^2).$$

In $(1,2)$: $J_F(1,2) = \begin{pmatrix} 2 & -4 \\ 4 & 2 \end{pmatrix}$ und $\det = 4 + 16 = 20$: Nahe $(1,2)$ streckt F kleine Flächen um den Faktor 20.

(b) $F(1,2) = (-3,\ 4)$ und $J_F(1,2)\,h = (2 \cdot 0{,}1 + 4 \cdot 0{,}1,\ 4 \cdot 0{,}1 - 2 \cdot 0{,}1) = (0{,}6,\ 0{,}2)$, also die Näherung $(-2{,}4;\ 4{,}2)$. Exakt ist $F(1{,}1;\ 1{,}9) = (1{,}21 - 3{,}61,\ 2 \cdot 1{,}1 \cdot 1{,}9) = (-2{,}4;\ 4{,}18)$, der Fehler also $(0;\ -0{,}02)$. Er ist genau $(h_1^2 - h_2^2,\ 2h_1h_2)$, und seine Länge $\|h\|^2$ ist klein gegen $\|h\|$.

(c) $F(1,2) = (-3,4)$ und $J_g(u,v) = (2u,\ 2v)$, an der Stelle $(-3,4)$ also $(-6,\ 8)$. Damit

$$J_{g\circ F}(1,2) = (-6,\ 8)\begin{pmatrix} 2 & -4 \\ 4 & 2 \end{pmatrix} = (-12+32,\ 24+16) = (20,\ 40).$$

Direkt: $g(F(x,y)) = (x^2+y^2)^2$, also $\nabla(g\circ F)(x,y) = 2\,(x^2+y^2)\cdot(2x,\ 2y) = 4\,(x^2+y^2)\,(x,\ y)$, in $(1,2)$ gleich $4 \cdot 5 \cdot (1,2) = (20,\ 40)$. Dasselbe.

(d) $J_P(r,\varphi) = \begin{pmatrix} \cos\varphi & -r\sin\varphi \\ \sin\varphi & r\cos\varphi \end{pmatrix}$, in $(2, \frac{\pi}{2})$ also $\begin{pmatrix} 0 & -2 \\ 1 & 0 \end{pmatrix}$ mit Determinante $2 = r$: Polarkoordinaten strecken Flächen um den Faktor r. Es ist $P(2, \frac{\pi}{2}) = (0,2)$ und $J_g(0,2) = (0,\ 4)$, also

$$J_{g\circ P}(2, \tfrac{\pi}{2}) = (0,\ 4)\begin{pmatrix} 0 & -2 \\ 1 & 0 \end{pmatrix} = (4,\ 0).$$

Direkt: $g(P(r,\varphi)) = r^2\cos^2\varphi + r^2\sin^2\varphi = r^2$ mit $J = (2r,\ 0) = (4,\ 0)$ in $(2, \frac{\pi}{2})$. Dasselbe.

Teil B: In Lean

Aufgabe B1 („Schließlich“: zu A1 (a))

$\forall^{\mathrm{f}}$ `x in F, P x` heißt „$P(x)$ gilt schließlich längs F“: Die Menge $\{x \mid P(x)\}$ liegt im Filter F. Längs `atTop` ist eine Menge groß, wenn sie ein Endstück $[N, \infty)$ enthält, längs $\mathcal{N}$ `a`, wenn sie eine Kugel um a enthält.

(a) Schließlich längs atTop ist $3x + 7 < x^2$ (A1 (a), ii).

(b) Schließlich längs $\mathcal{N}(2)$ ist $3 < x^2$ (A1 (a), iv).

(c) Gilt P schließlich und Q schließlich, und folgt R aus P und Q, so gilt R schließlich: die Filteraxiome (ii) und (iii) in einem Satz.

Hinweis: `eventually_ge_atTop N` ist das Endstück ab N als große Menge, `Metric.ball_mem_nhds a hε` die Kugel vom Radius ε um a. `filter_upwards [U, V] with x hx hy` macht aus dem Ziel „schließlich gilt …“ eine Aussage über einen Punkt x, an dem die großen Mengen U und V beide gelten – der Schnitt aus Axiom (iii). Ungleichungen mit Produkten erledigt `nlinarith`.

Lean lean/Blatt05.lean

```
-- ‘∀ᶠ x in F, P x‘ heisst: Die Menge {x | P x} ist gross, sie liegt im Filter F.
-- Laengs ‘atTop‘ ist gross, was ein Endstueck [N, ∞) enthaelt, laengs ‘𝒩 a‘,
-- was eine Kugel um a enthaelt.

/-- Aufgabe B1 (a) (A1 (a), ii): Schliesslich laengs ‘atTop‘ ist
    ‘3x + 7 < x²‘. -/
theorem schliesslich_quadrat : ∀ᶠ x : ℝ in atTop, 3 * x + 7 < x ^ 2 := by
  -- Nuetzlich: eventually_ge_atTop (das Endstueck [N, ∞)), filter_upwards,
    nlinarith
  sorry

/-- Aufgabe B1 (b) (A1 (a), iv): Schliesslich laengs ‘𝒩 2‘ ist
    ‘3 < x²‘. -/
theorem schliesslich_nahe_zwei : ∀ᶠ x : ℝ in 𝒩 2, 3 < x ^ 2 := by
  -- Nuetzlich: Metric.ball_mem_nhds (2 : ℝ) (die Kugel vom Radius 1/4),
  -- Metric.mem_ball, Real.dist_eq, abs_lt, nlinarith
  sorry

/-- Aufgabe B1 (c): Gilt ‘P‘ schliesslich und ‘Q‘ schliesslich, und folgt ‘R‘
    aus ‘P‘ und ‘Q‘, so gilt ‘R‘ schliesslich -- die Filteraxiome (ii) und
    (iii) in einem Satz. -/
theorem schliesslich_folgt {α : Type*} {F : Filter α} {P Q R : α → Prop}
    (hP : ∀ᶠ x in F, P x) (hQ : ∀ᶠ x in F, Q x) (h : ∀ x, P x → Q x → R x) :
    ∀ᶠ x in F, R x := by
  -- Nuetzlich: filter_upwards [hP, hQ] with x hp hq
  sorry
```

Lösung. (a) Das Endstück $[5, \infty)$ ist groß; `filter_upwards` lässt einen Punkt $x \geq 5$ übrig, und `nlinarith` multipliziert $x - 5 \geq 0$ mit sich selbst: $x^2 \geq 10x - 25 > 3x + 7$ für $x \geq 5$. (b) Die Kugel vom Radius $\frac{1}{4}$ um 2 ist groß; nach `abs_lt` steht $-\frac{1}{4} < x - 2 < \frac{1}{4}$ da, und `nlinarith` schließt aus $x > \frac{7}{4}$ auf $x^2 > \frac{49}{16} > 3$. (c) `filter_upwards [hP, hQ]` bildet den Schnitt der beiden großen Mengen (Axiom (iii)); in ihm gilt R nach der Annahme `h`, und die Menge, auf der R gilt, enthält ihn, ist also groß (Axiom (ii)).

Lean, Lösung loesungen/Blatt05_Loesung.lean

```
-- ‘∀ᶠ x in F, P x‘ heisst: Die Menge {x | P x} ist gross, sie liegt im Filter F.
-- Laengs ‘atTop‘ ist gross, was ein Endstueck [N, ∞) enthaelt, laengs ‘𝓝 a‘,
-- was eine Kugel um a enthaelt.

/-- Aufgabe B1 (a) (A1 (a), ii): Schliesslich laengs ‘atTop‘ ist
    ‘3x + 7 < x^2‘. -/
theorem schliesslich_quadrat : ∀ᶠ x : ℝ in atTop, 3 * x + 7 < x ^ 2 := by
  -- Nuetzlich: eventually_ge_atTop (das Endstueck [N, ∞)), filter_upwards,
    nlinarith
  filter_upwards [eventually_ge_atTop (5 : ℝ)] with x hx   -- das Endstueck [5,
    ∞)
  nlinarith                                                -- hx : 5 ≤ x

/-- Aufgabe B1 (b) (A1 (a), iv): Schliesslich laengs ‘𝓝 2‘ ist
    ‘3 < x^2‘. -/
theorem schliesslich_nahe_zwei : ∀ᶠ x : ℝ in 𝓝 2, 3 < x ^ 2 := by
  -- Nuetzlich: Metric.ball_mem_nhds (2 : ℝ) (die Kugel vom Radius 1/4),
  -- Metric.mem_ball, Real.dist_eq, abs_lt, nlinarith
  filter_upwards [Metric.ball_mem_nhds (2 : ℝ) (show (0 : ℝ) < 1 / 4 by
    norm_num)] with x hx
  rw [Metric.mem_ball, Real.dist_eq, abs_lt] at hx          -- -1/4 < x - 2 < 1/4
  nlinarith [hx.1, hx.2]

/-- Aufgabe B1 (c): Gilt ‘P‘ schliesslich und ‘Q‘ schliesslich, und folgt ‘R‘
    aus ‘P‘ und ‘Q‘, so gilt ‘R‘ schliesslich -- die Filteraxiome (ii) und
    (iii) in einem Satz. -/
theorem schliesslich_folgt {α : Type*} {F : Filter α} {P Q R : α → Prop}
    (hP : ∀ᶠ x in F, P x) (hQ : ∀ᶠ x in F, Q x) (h : ∀ x, P x → Q x → R x) :
    ∀ᶠ x in F, R x := by
  -- Nuetzlich: filter_upwards [hP, hQ] with x hp hq
  filter_upwards [hP, hQ] with x hp hq   -- der Schnitt {P} ∩ {Q}: Axiom (iii)
  exact h x hp hq                        -- darin gilt R: Axiom (ii)
```

Aufgabe B2 (Klein-o: zu A1 (c) und (d))

Die Definition `KleinO f g l` steht am Anfang des Lean-Kastens: zu jedem $c > 0$ schließlich $|f(x)| \le c\,|g(x)|$.

(a) $x^3 = o(x^2)$ längs $\mathcal{N}(0)$.

(b) $x^2 = o(x^3)$ längs atTop.

(c) x^3 ist längs atTop nicht $o(x^2)$.

(d) Die Linearkombination aus A1 (d): Sind $f_1 = o(g)$ und $f_2 = o(g)$ längs eines beliebigen Filters l und $a, b \in \mathbb{R}$, so ist $af_1 + bf_2 = o(g)$.

Hinweis: Die Beweise gehen wie die zu x^2 und x in der Vorlesung. Große Mengen liefern `Metric.ball_mem_nhds 0 hc` (die Kugel vom Radius c um 0) und `eventually_ge_atTop N` (das Endstück ab N). In (c) braucht man einen Punkt, an dem zwei Aussagen zugleich gelten; aus $\mathtt{h}_1$ und $\mathtt{h}_2$ macht $(\mathtt{h}_1\mathtt{.and}\ \mathtt{h}_2)\mathtt{.exists}$ ihn. In (d) bildet `filter_upwards` den Schnitt der beiden Mengen zu $c' = c/(|a| + |b| + 1)$. Ungleichungen mit Produkten erledigt `nlinarith`, wenn man ihm `sq_nonneg x` mitgibt.

Lean lean/Blatt05.lean

```
/-- Klein-o laengs eines Filters, wie in der Vorlesung: Zu jedem ‘c > 0‘ gilt
    schliesslich ‘|f x| ≤ c·|g x|‘. -/
def KleinO {α : Type*} (f g : α → ℝ) (l : Filter α) : Prop :=
  ∀ c > 0, ∀ᶠ x in l, |f x| ≤ c * |g x|

/-- Aufgabe B2 (a) (A1 (c), i): Bei ‘0‘ ist ‘x³‘ klein gegen ‘x²‘. -/
theorem kubik_klein_o_null :
    KleinO (fun x : ℝ => x ^ 3) (fun x => x ^ 2) (𝓝 0) := by
  -- Nuetzlich: wie quadrat_klein_o in der Vorlesung: intro c hc,
  -- filter_upwards [Metric.ball_mem_nhds (0 : ℝ) hc] with x hx, Metric.mem_ball,
  -- Real.dist_eq, sub_zero, abs_pow, positivity, nlinarith
  sorry

/-- Aufgabe B2 (b) (A1 (c), ii): Bei ‘+∞‘ ist es umgekehrt: ‘x²‘ ist
    klein gegen ‘x³‘. -/
theorem quadrat_klein_o_kubik :
    KleinO (fun x : ℝ => x ^ 2) (fun x => x ^ 3) atTop := by
  -- Nuetzlich: wie linear_klein_o_unendlich in der Vorlesung:
  -- filter_upwards [eventually_ge_atTop (1 / c)] with x hx, div_le_iff₀,
  -- positivity, abs_of_nonneg, sq_nonneg, pow_nonneg, nlinarith
  sorry

/-- Aufgabe B2 (c) (A1 (c), iii): Bei ‘+∞‘ ist ‘x³‘ nicht klein gegen
    ‘x²‘. -/
theorem kubik_nicht_klein_o :
    ¬ KleinO (fun x : ℝ => x ^ 3) (fun x => x ^ 2) atTop := by
  -- Nuetzlich: wie quadrat_nicht_klein_o_unendlich in der Vorlesung: c = 1
  -- (one_pos); Filter.Eventually.and und .exists liefern einen Punkt x, an
  -- dem die Ungleichung und x ≥ 2 (eventually_ge_atTop) zugleich gelten
  sorry

/-- Aufgabe B2 (d) (A1 (d)): Linearkombinationen von ‘o(g)‘ sind ‘o(g)‘ -- mit
    dem Schnitt zweier grosser Mengen. -/
theorem klein_o_linear {α : Type*} {l : Filter α} {f₁ f₂ g : α → ℝ} (a b : ℝ)
    (h₁ : KleinO f₁ g l) (h₂ : KleinO f₂ g l) :
    KleinO (fun x => a * f₁ x + b * f₂ x) g l := by
  -- Nuetzlich: wie klein_o_add in der Vorlesung, mit c / (|a| + |b| + 1)
  -- statt c / 2; positivity, div_pos, filter_upwards [h₁ _ _, h₂ _ _],
  -- abs_add_le, abs_mul, gcongr, div_le_one, calc
  sorry
```

Lösung. (a) Nach `intro c hc` ist zu zeigen, dass $|x^3| \le c\,|x^2|$ schließlich längs $\mathcal{N}(0)$ gilt. `filter_upwards` mit der Kugel vom Radius c lässt nur einen Punkt x mit $|x| < c$ übrig; nach dem Umschreiben ist $|x|^3 \le c\,|x|^2$ zu zeigen, und `nlinarith` multipliziert $|x| < c$ mit $|x|^2 \ge 0$ – die Rechnung von Papier. (b) Das Endstück ab $1/c$: Aus $x \ge 1/c$ wird mit $\texttt{div_le_iff}_0$ die Ungleichung $1 \le cx$, und $x^2 \le c\,x^3$ folgt aus $x^2\,(cx - 1) \ge 0$. (c) Angenommen, $x^3 = o(x^2)$ längs atTop. Mit $c = 1$ gilt schließlich $|x^3| \le |x^2|$ und schließlich auch $x \ge 2$; `.and` und `.exists` liefern ein x mit beidem – möglich, weil die leere Menge längs atTop nicht groß ist. Für $x \ge 2$ ist aber $x^3 \ge 2x^2 > x^2$. (d) Wie die Summenregel der Vorlesung, mit $c/(|a| + |b| + 1)$ statt $c/2$: `filter_upwards [h₁ _ hc', h₂ _ hc']` bildet den Schnitt der beiden großen Mengen, und die `calc`-Kette ist die Abschätzung aus A1 (d).

Lean, Lösung loesungen/Blatt05_Loesung.lean

```
/-- Klein-o laengs eines Filters, wie in der Vorlesung: Zu jedem ‘c > 0‘ gilt
    schliesslich ‘|f x| ≤ c·|g x|‘. -/
def KleinO {α : Type*} (f g : α → ℝ) (l : Filter α) : Prop :=
  ∀ c > 0, ∀ᶠ x in l, |f x| ≤ c * |g x|

/-- Aufgabe B2 (a) (A1 (c), i): Bei ‘0‘ ist ‘x³‘ klein gegen ‘x²‘. -/
theorem kubik_klein_o_null :
    KleinO (fun x : ℝ => x ^ 3) (fun x => x ^ 2) (𝓝 0) := by
  -- Nuetzlich: wie quadrat_klein_o in der Vorlesung: intro c hc,
  -- filter_upwards [Metric.ball_mem_nhds (0 : ℝ) hc] with x hx, Metric.mem_ball,
  -- Real.dist_eq, sub_zero, abs_pow, positivity, nlinarith
  intro c hc
  filter_upwards [Metric.ball_mem_nhds (0 : ℝ) hc] with x hx  -- (-c, c) ist
    gross
  rw [Metric.mem_ball, Real.dist_eq, sub_zero] at hx            -- hx : |x| < c
  rw [abs_pow, abs_pow]                                         -- |x|³ ≤ c·|x|²
  have h2 : 0 ≤ |x| ^ 2 := by positivity
  nlinarith [abs_nonneg x]

/-- Aufgabe B2 (b) (A1 (c), ii): Bei ‘+∞‘ ist es umgekehrt: ‘x²‘ ist
    klein gegen ‘x³‘. -/
theorem quadrat_klein_o_kubik :
    KleinO (fun x : ℝ => x ^ 2) (fun x => x ^ 3) atTop := by
  -- Nuetzlich: wie linear_klein_o_unendlich in der Vorlesung:
  -- filter_upwards [eventually_ge_atTop (1 / c)] with x hx, div_le_iff₀,
  -- positivity, abs_of_nonneg, sq_nonneg, pow_nonneg, nlinarith
  intro c hc
  filter_upwards [eventually_ge_atTop (1 / c)] with x hx      -- [1/c, ∞) ist
    gross
  have h1 : 1 ≤ c * x := by
    rwa [div_le_iff₀ hc, mul_comm] at hx
  have hx0 : 0 ≤ x := by
    have : 0 < 1 / c := by positivity
    linarith
  rw [abs_of_nonneg (sq_nonneg x), abs_of_nonneg (pow_nonneg hx0 3)]   -- x² ≤
    c·x³
  nlinarith [sq_nonneg x]

/-- Aufgabe B2 (c) (A1 (c), iii): Bei ‘+∞‘ ist ‘x³‘ nicht klein gegen
    ‘x²‘. -/
theorem kubik_nicht_klein_o :
    ¬ KleinO (fun x : ℝ => x ^ 3) (fun x => x ^ 2) atTop := by
  -- Nuetzlich: wie quadrat_nicht_klein_o_unendlich in der Vorlesung: c = 1
  -- (one_pos); Filter.Eventually.and und .exists liefern einen Punkt x, an
  -- dem die Ungleichung und x ≥ 2 (eventually_ge_atTop) zugleich gelten
  intro h
  obtain ⟨x, hx, h2⟩ := ((h 1 one_pos).and (eventually_ge_atTop (2 : ℝ))).exists
  have hx0 : (0 : ℝ) ≤ x := by linarith
  rw [abs_of_nonneg (pow_nonneg hx0 3), abs_of_nonneg (sq_nonneg x), one_mul] at
    hx
  nlinarith [sq_nonneg x]                                       -- hx : x³ ≤ x²

/-- Aufgabe B2 (d) (A1 (d)): Linearkombinationen von ‘o(g)‘ sind ‘o(g)‘ -- mit
    dem Schnitt zweier grosser Mengen. -/
theorem klein_o_linear {α : Type*} {l : Filter α} {f₁ f₂ g : α → ℝ} (a b : ℝ)
    (h₁ : KleinO f₁ g l) (h₂ : KleinO f₂ g l) :
    KleinO (fun x => a * f₁ x + b * f₂ x) g l := by
```

```
    -- Nuetzlich: wie klein_o_add in der Vorlesung, mit c / (|a| + |b| + 1)
    -- statt c / 2; positivity, div_pos, filter_upwards [h₁ _ _, h₂ _ _],
    -- abs_add_le, abs_mul, gcongr, div_le_one, calc
    intro c hc
    have hk : 0 < |a| + |b| + 1 := by positivity
    have hc' : 0 < c / (|a| + |b| + 1) := div_pos hc hk
    filter_upwards [h₁ _ hc', h₂ _ hc'] with x hx₁ hx₂              -- Schnitt:
      Axiom (iii)
    calc |a * f₁ x + b * f₂ x| ≤ |a| * |f₁ x| + |b| * |f₂ x| := by
          rw [← abs_mul, ← abs_mul]
          exact abs_add_le _ _
      _ ≤ |a| * (c / (|a| + |b| + 1) * |g x|) + |b| * (c / (|a| + |b| + 1) * |g
      x|) := by
          gcongr
      _ = (|a| + |b|) / (|a| + |b| + 1) * c * |g x| := by ring
      _ ≤ 1 * c * |g x| := by
          gcongr
          rw [div_le_one hk]
          linarith
      _ = c * |g x| := by ring
```

Aufgabe B3 (Die Ableitung: zu A2 (b) und (c))

Vorgegeben sind `quadrat_klein_o` ($(y-x)^2$ ist klein gegen $y-x$) und `ableitung_klein_o` (die Definition der Ableitung: Der Fehler der Tangente ist klein gegen $y-x$), beide wie in der Vorlesung.

(a) y^2+3y hat in x die Ableitung $2x+3$ – aus der Definition, wie y^2 in der Vorlesung.

(b) Der Betrag ist in 0 nicht klein gegen y, d. h. $|y| = o(y)$ längs $\mathcal{N}(0)$ gilt nicht. Das ist die Rechnung aus A2 (c) mit der Steigung 0.

(c) Der Betrag hat in 0 keine Ableitung.

Hinweis: In (a) ist der Fehler der Tangente $(y-x)^2$; `funext y` und `ring` zeigen das. In (b): `Metric.eventually_nhds_iff` macht aus „schließlich längs $\mathcal{N}(0)$“ eine Kugel vom Radius δ; der Punkt $\delta/2$ liegt darin. In (c) hat der Betrag in 0 ein lokales Minimum (`IsLocalMin`), nach Fermat – in Mathlib `IsLocalMin.hasDerivAt_eq_zero`, in der Vorlesung bewiesen – wäre die Ableitung also 0.

Lean lean/Blatt05.lean

```
-- Vorgegeben, wie in der Vorlesung: Bei ‘x‘ ist ‘(y - x)²‘ klein gegen ‘y - x‘,
-- und ‘f‘ hat in ‘x‘ die Ableitung ‘f'‘, wenn der Fehler der Tangente klein
-- gegen ‘y - x‘ ist.

/-- Bei ‘x‘ ist ‘(y - x)²‘ klein gegen ‘y - x‘: Zu jedem ‘c‘ reicht die Kugel
    vom Radius ‘c‘ um ‘x‘. -/
theorem quadrat_klein_o (x : ℝ) :
    KleinO (fun y : ℝ => (y - x) ^ 2) (fun y => y - x) (𝓝 x) := by
  intro c hc
  filter_upwards [Metric.ball_mem_nhds x hc] with y hy
  rw [Metric.mem_ball, Real.dist_eq] at hy
  rw [abs_pow, sq]
  exact mul_le_mul_of_nonneg_right hy.le (abs_nonneg _)

/-- Die Ableitung, wie auf der Folie: Der Fehler der Tangente ist klein gegen
    ‘y - x‘ laengs ‘𝓝 x‘. -/
theorem ableitung_klein_o (f : ℝ → ℝ) (f' x : ℝ) :
```

```
    HasDerivAt f f' x ↔
      KleinO (fun y => f y - f x - (y - x) * f') (fun y => y - x) (𝓝 x) := by
  rw [hasDerivAt_iff_isLittleO, Asymptotics.isLittleO_iff]
  simp only [KleinO, Real.norm_eq_abs, smul_eq_mul, gt_iff_lt]

/-- Aufgabe B3 (a) (A2 (b)): `y² + 3y` hat in `x` die Ableitung `2x + 3` --
    aus der Definition, wie `y²` in der Vorlesung. -/
theorem ableitung_quadrat_plus (x : ℝ) :
    HasDerivAt (fun y : ℝ => y ^ 2 + 3 * y) (2 * x + 3) x := by
  -- Nuetzlich: rw [ableitung_klein_o]; der Fehler der Tangente ist (y - x)^2
  -- (have fehler : (fun y => ...) = fun y => (y - x) ^ 2, ext y, ring);
  -- dann quadrat_klein_o
  sorry

/-- Aufgabe B3 (b) (A2 (c)): Der Betrag ist in `0` nicht klein gegen `y`:
    Mit `c = 1/2` muesste schliesslich `|y| ≤ (1/2)·|y|` gelten. -/
theorem betrag_nicht_klein_o :
    ¬ KleinO (fun y : ℝ => |y|) (fun y => y) (𝓝 0) := by
  -- Nuetzlich: intro h; h (1 / 2) (by norm_num); Metric.eventually_nhds_iff
  -- liefert ein δ > 0; der Punkt δ / 2 liegt in der Kugel (Real.dist_eq,
  -- abs_of_pos, half_pos) und verletzt die Ungleichung; abs_abs, linarith
  sorry

/-- Aufgabe B3 (c) (A2 (c)): Der Betrag hat in `0` keine Ableitung. -/
theorem betrag_keine_ableitung :
    ¬ ∃ f' : ℝ, HasDerivAt (fun y : ℝ => |y|) f' 0 := by
  -- Nuetzlich: In 0 liegt ein lokales Minimum (IsLocalMin, Eventually.of_forall),
  -- also ist f' = 0 nach Fermat: IsLocalMin.hasDerivAt_eq_zero. Dann
  -- ableitung_klein_o und (b); simpa
  sorry
```

Lösung. (a) `rw [ableitung_klein_o]` macht aus der Aussage über die Ableitung die über den Fehler der Tangente. Der Fehler ist $(y-x)^2$, das zeigt `ring`; `quadrat_klein_o` schließt das Ziel. (b) Angenommen, $|y| = o(y)$. Mit $c = \frac{1}{2}$ gilt dann schließlich $||y|| \le \frac{1}{2}|y|$, auf einer Kugel vom Radius δ. Im Punkt $\delta/2$ steht dann $\frac{\delta}{2} \le \frac{\delta}{4}$ – falsch. (c) Wäre f' eine Ableitung des Betrags in 0, so wäre $f' = 0$ nach Fermat, denn in 0 liegt ein lokales Minimum. Mit $f' = 0$ ist der Fehler der Tangente aber $|y|$, und das ist nach (b) nicht klein gegen y. Das ist A2 (c): Die Sekantensteigungen ± 1 haben keinen Grenzwert.

Lean, Lösung loesungen/Blatt05_Loesung.lean

```
-- Vorgegeben, wie in der Vorlesung: Bei `x` ist `(y - x)²` klein gegen `y - x`,
-- und `f` hat in `x` die Ableitung `f'`, wenn der Fehler der Tangente klein
-- gegen `y - x` ist.

/-- Bei `x` ist `(y - x)²` klein gegen `y - x`: Zu jedem `c` reicht die Kugel
    vom Radius `c` um `x`. -/
theorem quadrat_klein_o (x : ℝ) :
    KleinO (fun y : ℝ => (y - x) ^ 2) (fun y => y - x) (𝓝 x) := by
  intro c hc
  filter_upwards [Metric.ball_mem_nhds x hc] with y hy
  rw [Metric.mem_ball, Real.dist_eq] at hy
  rw [abs_pow, sq]
  exact mul_le_mul_of_nonneg_right hy.le (abs_nonneg _)

/-- Die Ableitung, wie auf der Folie: Der Fehler der Tangente ist klein gegen
    `y - x` laengs `𝓝 x`. -/
theorem ableitung_klein_o (f : ℝ → ℝ) (f' x : ℝ) :
```

```
    HasDerivAt f f' x ↔
      KleinO (fun y => f y - f x - (y - x) * f') (fun y => y - x) (𝓝 x) := by
  rw [hasDerivAt_iff_isLittleO, Asymptotics.isLittleO_iff]
  simp only [KleinO, Real.norm_eq_abs, smul_eq_mul, gt_iff_lt]

/-- Aufgabe B3 (a) (A2 (b)): `y² + 3y` hat in `x` die Ableitung `2x + 3` --
    aus der Definition, wie `y²` in der Vorlesung. -/
theorem ableitung_quadrat_plus (x : ℝ) :
    HasDerivAt (fun y : ℝ => y ^ 2 + 3 * y) (2 * x + 3) x := by
  -- Nuetzlich: rw [ableitung_klein_o]; der Fehler der Tangente ist (y - x)^2
  -- (have fehler : (fun y => ...) = fun y => (y - x) ^ 2, ext y, ring);
  -- dann quadrat_klein_o
  rw [ableitung_klein_o]
  have fehler : (fun y : ℝ => (y ^ 2 + 3 * y) - (x ^ 2 + 3 * x) - (y - x) * (2 *
    x + 3)) =
      fun y => (y - x) ^ 2 := by
    funext y
    ring
  rw [fehler]                              -- Ziel: (y - x)^2 ist klein gegen y - x
  exact quadrat_klein_o x

/-- Aufgabe B3 (b) (A2 (c)): Der Betrag ist in `0` nicht klein gegen `y`:
    Mit `c = 1/2` muesste schliesslich `|y| ≤ (1/2)·|y|` gelten. -/
theorem betrag_nicht_klein_o :
    ¬ KleinO (fun y : ℝ => |y|) (fun y => y) (𝓝 0) := by
  -- Nuetzlich: intro h; h (1 / 2) (by norm_num); Metric.eventually_nhds_iff
  -- liefert ein δ > 0; der Punkt δ / 2 liegt in der Kugel (Real.dist_eq,
  -- abs_of_pos, half_pos) und verletzt die Ungleichung; abs_abs, linarith
  intro h
  have h1 := h (1 / 2) (by norm_num)               -- schliesslich |(|y|)| ≤
    (1/2)·|y|
  rw [Metric.eventually_nhds_iff] at h1
  obtain ⟨δ, hδ, h2⟩ := h1                         -- gilt auf der Kugel vom
    Radius δ
  have hd : δ / 2 > 0 := half_pos hδ
  have h3 := h2 (y := δ / 2) (by
    rw [Real.dist_eq, sub_zero, abs_of_pos hd]     -- δ/2 liegt in der Kugel
    linarith)
  rw [abs_abs, abs_of_pos hd] at h3                -- h3 : δ/2 ≤ (1/2)·(δ/2)
  linarith

/-- Aufgabe B3 (c) (A2 (c)): Der Betrag hat in `0` keine Ableitung. -/
theorem betrag_keine_ableitung :
    ¬ ∃ f' : ℝ, HasDerivAt (fun y : ℝ => |y|) f' 0 := by
  -- Nuetzlich: In 0 liegt ein lokales Minimum (IsLocalMin,
    Eventually.of_forall),
  -- also ist f' = 0 nach Fermat: IsLocalMin.hasDerivAt_eq_zero. Dann
  -- ableitung_klein_o und (b); simpa
  rintro ⟨f', hf⟩
  have hmin : IsLocalMin (fun y : ℝ => |y|) 0 :=
    Eventually.of_forall fun y => by simp          -- |0| ≤ |y|
  have h0 : f' = 0 := hmin.hasDerivAt_eq_zero hf   -- Fermat
  subst h0
  rw [ableitung_klein_o] at hf                     -- der Fehler der
    Tangente 0
  exact betrag_nicht_klein_o (by simpa using hf)
```

Aufgabe B4 (Partielle Ableitungen: zu A3 (a) und (d))

Punkte der Ebene sind in diesem Teil Vektoren `v : Fin 2 → ℝ` mit den Koordinaten `v 0` und `v 1`. `HatPartielleAbleitung f x j a` heißt: Die Schnittkurve $t \mapsto f(x + t\,e_j)$ hat in 0 die Ableitung a. Das Beispiel x^2y nach x aus der Vorlesung steht als `example` in der Datei.

(a) $f(x, y) = x^2y + 3y^2$ hat in v nach y die partielle Ableitung $x^2 + 6y$.

(b) Das Gegenbeispiel $k(x, y) = \frac{xy}{x^2+y^2}$ aus A3 (d) ist in Lean `gegenbeispiel`, mit $k(0, 0) = 0$, weil in Lean $a/0 = 0$ ist. Seine partielle Ableitung nach y in 0 ist 0.

(c) Auf der Diagonalen ist $k(t, t) = \frac{1}{2}$ für $t \neq 0$.

Hinweis: (a) geht wie das Beispiel in der Datei: `unfold HatPartielleAbleitung`, dann die Schnittkurve ausrechnen (`funext t`, `simp`), dann die Regeln `hasDerivAt_id`, `.const_add`, `.pow`, `.const_mul` und `.add`, und zuletzt `convert … using 1`. (b) geht wie die partielle Ableitung nach x in der Vorlesung: Die Schnittkurve ist die Nullfunktion, `hasDerivAt_const`. (c) `field_simp` und `ring`.

Lean `lean/Blatt05.lean`

```
/-- Partielle Ableitung, wie in der Vorlesung: ‘f‘ hat in ‘x‘ nach der
    ‘j‘-ten Koordinate die partielle Ableitung ‘a‘, wenn die Schnittkurve
    ‘t ↦ f(x + t·e_j)‘ in ‘0‘ die Ableitung ‘a‘ hat. Punkte der Ebene sind
    Vektoren ‘v : Fin 2 → ℝ‘ mit den Koordinaten ‘v 0‘ und ‘v 1‘. -/
def HatPartielleAbleitung {n : ℕ} {F : Type*} [NormedAddCommGroup F] [NormedSpace ℝ
    F]
    (f : (Fin n → ℝ) → F) (x : Fin n → ℝ) (j : Fin n) (a : F) : Prop :=
  HasDerivAt (fun t : ℝ => f (x + t • (Pi.single j 1 : Fin n → ℝ))) a 0

/-- Vorgegeben: das Beispiel der Vorlesung, ‘f(x, y) = x²y‘ nach ‘x‘. -/
example (v : Fin 2 → ℝ) :
    HatPartielleAbleitung (fun w : Fin 2 → ℝ => w 0 ^ 2 * w 1) v 0 (2 * v 0 * v 1)
    := by
  unfold HatPartielleAbleitung
  have e : (fun t : ℝ => (v + t • (Pi.single 0 1 : Fin 2 → ℝ)) 0 ^ 2 *
      (v + t • (Pi.single 0 1 : Fin 2 → ℝ)) 1) = fun t => (v 0 + t) ^ 2 * v 1 := by
    funext t
    simp
  rw [e]
  have h := (((hasDerivAt_id (0 : ℝ)).const_add (v 0)).pow 2).mul_const (v 1)
  convert h using 1
  simp

/-- Aufgabe B4 (a) (A3 (a)): ‘f(x, y) = x²y + 3y²‘ hat in ‘v‘ die partielle
    Ableitung ‘x² + 6y‘ nach ‘y‘. -/
theorem partiell_nach_y (v : Fin 2 → ℝ) :
    HatPartielleAbleitung (fun w : Fin 2 → ℝ => w 0 ^ 2 * w 1 + 3 * w 1 ^ 2) v 1
      (v 0 ^ 2 + 6 * v 1) := by
  -- Nuetzlich: wie das Beispiel x²y oben: unfold HatPartielleAbleitung; die
  -- Schnittkurve ausrechnen (funext t, simp); hasDerivAt_id, .const_add, .pow,
  -- .const_mul, .add; convert ... using 1; simp; ring
  sorry

/-- In Lean ist ‘a / 0 = 0‘, also ‘f(0, 0) = 0‘ von selbst. -/
noncomputable def gegenbeispiel (w : Fin 2 → ℝ) : ℝ :=
  w 0 * w 1 / (w 0 ^ 2 + w 1 ^ 2)

/-- Aufgabe B4 (b) (A3 (d)): Auf der ‘y‘-Achse ist ‘f‘ null; die partielle
    Ableitung nach ‘y‘ in ‘0‘ ist ‘0‘. -/
theorem gegenbeispiel_partiell_y : HatPartielleAbleitung gegenbeispiel 0 1 0 := by
```

```
  -- Nuetzlich: wie die partielle Ableitung nach x in der Vorlesung: Die
  -- Schnittkurve ist die Nullfunktion (funext t, simp [gegenbeispiel]);
  -- hasDerivAt_const
  sorry

/-- Aufgabe B4 (c) (A3 (d)): Auf der Diagonalen ist `f = 1/2`, ausser in `0`. -/
theorem gegenbeispiel_diagonale {t : ℝ} (ht : t ≠ 0) :
    gegenbeispiel ![t, t] = 1 / 2 := by
  -- Nuetzlich: simp only [gegenbeispiel, Matrix.cons_val_zero,
    Matrix.cons_val_one],
  -- field_simp, ring
  sorry
```

Lösung. (a) `unfold` setzt die Definition ein, `funext t` und `simp` rechnen die Schnittkurve aus: x bleibt, y wird $v_1 + t$, die Kurve ist $t \mapsto v_0^2\,(v_1 + t) + 3\,(v_1 + t)^2$. $v_1 + t$ hat die Ableitung 1; `.const_mul`, `.pow` und `.add` setzen die Regeln zusammen, und `convert … using 1` lässt die Gleichung der Ableitungswerte übrig, die `simp` und `ring` nachrechnen: $v_0^2 \cdot 1 + 3 \cdot 2\, v_1 \cdot 1 = v_0^2 + 6 v_1$. (b) Auf der y-Achse ist $x = 0$ und damit $k(0, t) = \frac{0}{t^2} = 0$ (auch für $t = 0$): Die Schnittkurve ist die Nullfunktion, und eine konstante Funktion hat die Ableitung 0. (c) `field_simp` multipliziert den Nenner weg ($t \neq 0$, also auch $t^2 + t^2 \neq 0$); übrig bleibt eine Gleichung zwischen Polynomen in t, die `ring` nachrechnet. Zusammen mit (b) und der Vorlesung: k hat in 0 beide partiellen Ableitungen, ist dort aber nicht stetig.

Lean, Lösung loesungen/Blatt05_Loesung.lean

```
/-- Partielle Ableitung, wie in der Vorlesung: `f` hat in `x` nach der
    `j`-ten Koordinate die partielle Ableitung `a`, wenn die Schnittkurve
    `t ↦ f(x + t·e_j)` in `0` die Ableitung `a` hat. Punkte der Ebene sind
    Vektoren `v : Fin 2 → ℝ` mit den Koordinaten `v 0` und `v 1`. -/
def HatPartielleAbleitung {n : ℕ} {F : Type*} [NormedAddCommGroup F]
    [NormedSpace ℝ F]
    (f : (Fin n → ℝ) → F) (x : Fin n → ℝ) (j : Fin n) (a : F) : Prop :=
  HasDerivAt (fun t : ℝ => f (x + t • (Pi.single j 1 : Fin n → ℝ))) a 0

/-- Vorgegeben: das Beispiel der Vorlesung, `f(x, y) = x²y` nach `x`. -/
example (v : Fin 2 → ℝ) :
    HatPartielleAbleitung (fun w : Fin 2 → ℝ => w 0 ^ 2 * w 1) v 0 (2 * v 0 * v
    1) := by
  unfold HatPartielleAbleitung
  have e : (fun t : ℝ => (v + t • (Pi.single 0 1 : Fin 2 → ℝ)) 0 ^ 2 *
      (v + t • (Pi.single 0 1 : Fin 2 → ℝ)) 1) = fun t => (v 0 + t) ^ 2 * v 1
    := by
    funext t
    simp
  rw [e]
  have h := (((hasDerivAt_id (0 : ℝ)).const_add (v 0)).pow 2).mul_const (v 1)
  convert h using 1
  simp

/-- Aufgabe B4 (a) (A3 (a)): `f(x, y) = x²y + 3y²` hat in `v` die partielle
    Ableitung `x² + 6y` nach `y`. -/
theorem partiell_nach_y (v : Fin 2 → ℝ) :
    HatPartielleAbleitung (fun w : Fin 2 → ℝ => w 0 ^ 2 * w 1 + 3 * w 1 ^ 2) v 1
      (v 0 ^ 2 + 6 * v 1) := by
  -- Nuetzlich: wie das Beispiel x²y oben: unfold HatPartielleAbleitung; die
  -- Schnittkurve ausrechnen (funext t, simp); hasDerivAt_id, .const_add, .pow,
  -- .const_mul, .add; convert … using 1; simp; ring
  unfold HatPartielleAbleitung
```

```
  -- Die Schnittkurve: x bleibt, y wird v₁ + t.
  have e : (fun t : ℝ => (v + t • (Pi.single 1 1 : Fin 2 → ℝ)) 0 ^ 2 *
      (v + t • (Pi.single 1 1 : Fin 2 → ℝ)) 1 +
      3 * (v + t • (Pi.single 1 1 : Fin 2 → ℝ)) 1 ^ 2) =
      fun t => v 0 ^ 2 * (v 1 + t) + 3 * (v 1 + t) ^ 2 := by
    funext t
    simp
  rw [e]
  -- v₁ + t hat die Ableitung 1; dann Faktor, Quadrat und Summe.
  have h1 : HasDerivAt (fun t : ℝ => v 1 + t) 1 0 := (hasDerivAt_id (0 : ℝ
    )).const_add (v 1)
  have h := (h1.const_mul (v 0 ^ 2)).add ((h1.pow 2).const_mul 3)
  convert h using 1
  simp
  ring

/-- In Lean ist `a / 0 = 0`, also `f(0, 0) = 0` von selbst. -/
noncomputable def gegenbeispiel (w : Fin 2 → ℝ) : ℝ :=
  w 0 * w 1 / (w 0 ^ 2 + w 1 ^ 2)

/-- Aufgabe B4 (b) (A3 (d)): Auf der `y`-Achse ist `f` null; die partielle
    Ableitung nach `y` in `0` ist `0`. -/
theorem gegenbeispiel_partiell_y : HatPartielleAbleitung gegenbeispiel 0 1 0 :=
    by
  -- Nuetzlich: wie die partielle Ableitung nach x in der Vorlesung: Die
  -- Schnittkurve ist die Nullfunktion (funext t, simp [gegenbeispiel]);
  -- hasDerivAt_const
  unfold HatPartielleAbleitung
  have h : (fun t : ℝ => gegenbeispiel (0 + t • (Pi.single 1 1 : Fin 2 → ℝ))) =
      fun _ => 0 := by
    funext t
    simp [gegenbeispiel]
  rw [h]
  exact hasDerivAt_const 0 0

/-- Aufgabe B4 (c) (A3 (d)): Auf der Diagonalen ist `f = 1/2`, ausser in `0`. -/
theorem gegenbeispiel_diagonale {t : ℝ} (ht : t ≠ 0) :
    gegenbeispiel ![t, t] = 1 / 2 := by
  -- Nuetzlich: simp only [gegenbeispiel, Matrix.cons_val_zero,
    Matrix.cons_val_one],
  -- field_simp, ring
  simp only [gegenbeispiel, Matrix.cons_val_zero, Matrix.cons_val_one]
  field_simp
  ring
```

Aufgabe B5 (Jacobi-Matrix und Kettenregel: zu A4 (a) und (c))

Punkte der Ebene sind Vektoren `v : Fin 2 → ℝ` mit den Koordinaten `v 0` und `v 1`; `![a, b]` ist der Vektor (a, b), `!![a, b; c, d]` die Matrix mit den Zeilen (a, b) und (c, d). Vorgegeben sind wie in der Vorlesung `matrixAbb`, das aus einer Matrix die lineare Abbildung $h \mapsto Mh$ macht, das Lemma `matrixAbb_comp` (Hintereinanderausführen ist Matrizenmultiplizieren) und `jacobi_quadratsumme`, die Jacobi-Matrix $(2u,\ 2v)$ von $g(u, v) = u^2 + v^2$.

(a) $F(x, y) = (x^2 - y^2,\ 2xy)$ hat die Jacobi-Matrix aus A4 (a).

(b) Kettenregel: $g \circ F$ hat die Jacobi-Matrix $J_g(F(v)) \cdot J_F(v)$.

(c) Das Produkt ausgerechnet ist $4\,(x^2+y^2)\,(x,\ y)$ – der Gradient aus A4 (c).

Hinweis: (a) geht wie `jacobi_xy` in der Vorlesung: `hasFDerivAt_pi'` zerlegt die Aussage in eine je Zeile, `fin_cases i` unterscheidet die Zeilen, `hasFDerivAt_apply 0 v` ist die Ableitung der Koordinate `w 0`; dazu `.pow`, `.sub`, `.const_mul` und `.mul`. (b) wie `kettenregel_polar` in der Vorlesung. (c) `ext i j` und `fin_cases` gehen die Einträge durch; `simp` mit `Matrix.mul_apply` und `Fin.sum_univ_two` rechnet sie aus.

Lean `lean/Blatt05.lean`

```
-- Vorgegeben, wie in der Vorlesung: eine Matrix als lineare Abbildung, das
-- Hintereinanderausfuehren als Matrixprodukt und die Jacobi-Matrix von
-- u² + v². Punkte sind Vektoren ‘v : Fin 2 → ℝ‘ mit ‘v 0 = x‘, ‘v 1 = y‘.

/-- Eine m×n-Matrix als lineare Abbildung ℝ^n → ℝ^m, h ↦ M h. -/
noncomputable def matrixAbb {m n : ℕ} (M : Matrix (Fin m) (Fin n) ℝ) :
    (Fin n → ℝ) →L[ℝ] (Fin m → ℝ) :=
  LinearMap.toContinuousLinearMap (Matrix.toLin' M)

/-- Angewandt auf h ist sie Matrix mal Vektor. -/
@[simp] lemma matrixAbb_apply {m n : ℕ} (M : Matrix (Fin m) (Fin n) ℝ)
    (h : Fin n → ℝ) : matrixAbb M h = M.mulVec h := rfl

/-- Hintereinander ausfuehren heisst Matrizen multiplizieren. -/
lemma matrixAbb_comp {m n k : ℕ} (A : Matrix (Fin m) (Fin n) ℝ)
    (B : Matrix (Fin n) (Fin k) ℝ) :
    (matrixAbb A).comp (matrixAbb B) = matrixAbb (A * B) := by
  ext h : 1
  simp [Matrix.mulVec_mulVec]

/-- g(u, v) = u² + v², als Abbildung nach ℝ¹: Jacobi-Matrix (2u, 2v). -/
theorem jacobi_quadratsumme (u : Fin 2 → ℝ) :
    HasFDerivAt (fun w : Fin 2 → ℝ => ![w 0 ^ 2 + w 1 ^ 2])
      (matrixAbb !![2 * u 0, 2 * u 1]) u := by
  rw [hasFDerivAt_pi']
  intro i
  fin_cases i
  have h := ((hasFDerivAt_apply (𝕜 := ℝ) 0 u).pow 2).add
    ((hasFDerivAt_apply (𝕜 := ℝ) 1 u).pow 2)
  convert h using 1
  ext k
  simp [dotProduct, Fin.sum_univ_two]

/-- Aufgabe B5 (a) (A4 (a)): ‘F(x, y) = (x² - y², 2xy)‘ hat die Jacobi-Matrix
    ‘[[2x, -2y], [2y, 2x]]‘. -/
theorem jacobi_quadrat (v : Fin 2 → ℝ) :
    HasFDerivAt (fun w : Fin 2 → ℝ => ![w 0 ^ 2 - w 1 ^ 2, 2 * w 0 * w 1])
      (matrixAbb !![2 * v 0, -(2 * v 1); 2 * v 1, 2 * v 0]) v := by
  -- Nuetzlich: wie jacobi_xy in der Vorlesung: rw [hasFDerivAt_pi'], intro i,
  -- fin_cases i, hasFDerivAt_apply (𝕜 := ℝ) 0 v, HasFDerivAt.pow, .sub,
  -- .const_mul, .mul, convert ... using 1, ext k,
  -- simp [dotProduct, Fin.sum_univ_two], ring
  sorry

/-- Aufgabe B5 (b) (A4 (c), Kettenregel): ‘g ∘ F‘ hat die Jacobi-Matrix
    ‘J_g(F(v)) · J_F(v)‘. -/
theorem kette_quadrat (v : Fin 2 → ℝ) :
    HasFDerivAt (fun w : Fin 2 → ℝ => ![(w 0 ^ 2 - w 1 ^ 2) ^ 2 + (2 * w 0 * w 1)
    ^ 2])
      (matrixAbb (!![2 * (v 0 ^ 2 - v 1 ^ 2), 2 * (2 * v 0 * v 1)] *
```

```
        !![2 * v 0, -(2 * v 1); 2 * v 1, 2 * v 0])) v := by
  -- Nuetzlich: wie kettenregel_polar in der Vorlesung: HasFDerivAt.comp mit
  -- jacobi_quadratsumme an der Stelle ![v 0 ^ 2 - v 1 ^ 2, 2 * v 0 * v 1] und
  -- jacobi_quadrat v; matrixAbb_comp; simpa
  sorry

/-- Aufgabe B5 (c) (A4 (c), Rechnung): Das Produkt der Jacobi-Matrizen ist
    `4(x² + y²)·(x, y)` -- der Gradient von `(x² + y²)²`. -/
theorem produkt_quadrat (x y : ℝ) :
    !![2 * (x ^ 2 - y ^ 2), 2 * (2 * x * y)] * !![2 * x, -(2 * y); 2 * y, 2 * x] =
      !![4 * x * (x ^ 2 + y ^ 2), 4 * y * (x ^ 2 + y ^ 2)] := by
  -- Nuetzlich: ext i j, fin_cases i, fin_cases j,
  -- simp [Matrix.mul_apply, Fin.sum_univ_two], ring
  sorry
```

Lösung. (a) Nach `rw [hasFDerivAt_pi']`, `intro i` und `fin_cases i` ist je Zeile eine Ableitung zu zeigen. Die erste Zeile $x^2 - y^2$ setzen `.pow 2` und `.sub` zusammen, die zweite $2xy$ setzen `.const_mul 2` und `.mul` zusammen. `convert ... using 1` lässt die Gleichheit zweier linearer Abbildungen übrig, und `ext k` prüft sie an den Einheitsvektoren; dort ist es eine Rechnung, die `simp` und `ring` erledigen. (b) `HasFDerivAt.comp` setzt `jacobi_quadratsumme` an der Stelle $F(v)$ und (a) zusammen; `matrixAbb_comp` macht aus dem Hintereinanderausführen das Matrixprodukt, und `simpa` gleicht die Funktionsterme an. (c) Das Produkt ist eine 1×2-Matrix; nach `simp` ist jeder ihrer zwei Einträge eine Polynomgleichung für `ring` – die Rechnung aus A4 (c). Die Determinante aus A4 (a) ist `Matrix.det_fin_two_of` und `ring`.

Lean, Lösung loesungen/Blatt05_Loesung.lean

```
-- Vorgegeben, wie in der Vorlesung: eine Matrix als lineare Abbildung, das
-- Hintereinanderausfuehren als Matrixprodukt und die Jacobi-Matrix von
-- u² + v². Punkte sind Vektoren `v : Fin 2 → ℝ` mit `v 0 = x`, `v 1 = y`.

/-- Eine m×n-Matrix als lineare Abbildung ℝⁿ → ℝᵐ, h ↦ M h. -/
noncomputable def matrixAbb {m n : ℕ} (M : Matrix (Fin m) (Fin n) ℝ) :
    (Fin n → ℝ) →L[ℝ] (Fin m → ℝ) :=
  LinearMap.toContinuousLinearMap (Matrix.toLin' M)

/-- Angewandt auf h ist sie Matrix mal Vektor. -/
@[simp] lemma matrixAbb_apply {m n : ℕ} (M : Matrix (Fin m) (Fin n) ℝ)
    (h : Fin n → ℝ) : matrixAbb M h = M.mulVec h := rfl

/-- Hintereinander ausfuehren heisst Matrizen multiplizieren. -/
lemma matrixAbb_comp {m n k : ℕ} (A : Matrix (Fin m) (Fin n) ℝ)
    (B : Matrix (Fin n) (Fin k) ℝ) :
    (matrixAbb A).comp (matrixAbb B) = matrixAbb (A * B) := by
  ext h : 1
  simp [Matrix.mulVec_mulVec]

/-- g(u, v) = u² + v², als Abbildung nach ℝ¹: Jacobi-Matrix (2u, 2v). -/
theorem jacobi_quadratsumme (u : Fin 2 → ℝ) :
    HasFDerivAt (fun w : Fin 2 → ℝ => ![w 0 ^ 2 + w 1 ^ 2])
      (matrixAbb !![2 * u 0, 2 * u 1]) u := by
  rw [hasFDerivAt_pi']
  intro i
  fin_cases i
  have h := ((hasFDerivAt_apply (𝕜 := ℝ) 0 u).pow 2).add
    ((hasFDerivAt_apply (𝕜 := ℝ) 1 u).pow 2)
  convert h using 1
  ext k
```

```
  simp [dotProduct, Fin.sum_univ_two]

/-- Aufgabe B5 (a) (A4 (a)): 'F(x, y) = (x² - y², 2xy)' hat die Jacobi-Matrix
    '[[2x, -2y], [2y, 2x]]'. -/
theorem jacobi_quadrat (v : Fin 2 → ℝ) :
    HasFDerivAt (fun w : Fin 2 → ℝ => ![w 0 ^ 2 - w 1 ^ 2, 2 * w 0 * w 1])
      (matrixAbb ![![2 * v 0, -(2 * v 1)], ![2 * v 1, 2 * v 0]]) v := by
  -- Nuetzlich: wie jacobi_xy in der Vorlesung: rw [hasFDerivAt_pi'], intro i,
  -- fin_cases i, hasFDerivAt_apply (𝕜 := ℝ) 0 v, HasFDerivAt.pow, .sub,
  -- .const_mul, .mul, convert ... using 1, ext k,
  -- simp [dotProduct, Fin.sum_univ_two], ring
  rw [hasFDerivAt_pi']
  intro i
  fin_cases i
  · -- erste Zeile: x² - y²
    have h := ((hasFDerivAt_apply (𝕜 := ℝ) 0 v).pow 2).sub
      ((hasFDerivAt_apply (𝕜 := ℝ) 1 v).pow 2)
    convert h using 1
    ext k
    simp [dotProduct, Fin.sum_univ_two]
    ring
  · -- zweite Zeile: 2xy
    have h := ((hasFDerivAt_apply (𝕜 := ℝ) 0 v).const_mul 2).mul
      (hasFDerivAt_apply (𝕜 := ℝ) 1 v)
    convert h using 1
    ext k
    simp [dotProduct, Fin.sum_univ_two]
    ring

/-- Aufgabe B5 (b) (A4 (c), Kettenregel): 'g ∘ F' hat die Jacobi-Matrix
    'J_g(F(v)) · J_F(v)'. -/
theorem kette_quadrat (v : Fin 2 → ℝ) :
    HasFDerivAt (fun w : Fin 2 → ℝ => ![(w 0 ^ 2 - w 1 ^ 2) ^ 2 + (2 * w 0 * w
    1) ^ 2])
      (matrixAbb (![![2 * (v 0 ^ 2 - v 1 ^ 2), 2 * (2 * v 0 * v 1)]] *
        ![![2 * v 0, -(2 * v 1)], ![2 * v 1, 2 * v 0]])) v := by
  -- Nuetzlich: wie kettenregel_polar in der Vorlesung: HasFDerivAt.comp mit
  -- jacobi_quadratsumme an der Stelle ![v 0 ^ 2 - v 1 ^ 2, 2 * v 0 * v 1] und
  -- jacobi_quadrat v; matrixAbb_comp; simpa
  have h := (jacobi_quadratsumme ![v 0 ^ 2 - v 1 ^ 2, 2 * v 0 * v 1]).comp v
    (jacobi_quadrat v)
  rw [matrixAbb_comp] at h
  simpa using h

/-- Aufgabe B5 (c) (A4 (c), Rechnung): Das Produkt der Jacobi-Matrizen ist
    '4(x² + y²)·(x, y)' -- der Gradient von '(x² + y²)²'. -/
theorem produkt_quadrat (x y : ℝ) :
    ![![2 * (x ^ 2 - y ^ 2), 2 * (2 * x * y)]] * ![![2 * x, -(2 * y)], ![2 * y, 2 * x]]
    =
      ![![4 * x * (x ^ 2 + y ^ 2), 4 * y * (x ^ 2 + y ^ 2)]] := by
  -- Nuetzlich: ext i j, fin_cases i, fin_cases j,
  -- simp [Matrix.mul_apply, Fin.sum_univ_two], ring
  ext i j
  fin_cases i
  fin_cases j <;> simp [Matrix.mul_apply, Fin.sum_univ_two] <;> ring
```